

1. PROBLEMY PRZED ZMIANĄ:

- a) Projekt był podzielony na dwa osobne bitstreamy (gałęzie master_new i player_2_new), z których tylko jeden (master_new) generował obraz VGA i prowadził rozgrywkę - druga płytką była zredukowana do kontrolera przycisków bez własnego ekranu.
- b) Komunikacja UART była jednokierunkowa: płytką master_new miała tylko wejście RX, bez nadajnika TX.
- c) Dwie gałęzie w repozytorium oznaczały, że po sklonowaniu i zbudowaniu gałęzi domyślnej powstawała tylko jedna z dwóch konfiguracji.

2. WPROWADZONA ZMIANA ARCHITEKTURY

Zamiast dwóch różnych projektów - jeden, identyczny bitstream wgrywany na OBIĘ płytki Basys3. Rolę (HOST / Gracz 1 albo CLIENT / Gracz 2) wybiera fizyczny przełącznik (**SW0**) sw_role, odczytywany w locie:

- HOST (sw_role = 0): jedyna płytką licząca fizykę piłki i wynik (moduł ball_pos). Wysyła do drugiej płytki pełny stan gry.
- CLIENT (sw_role = 1): nie liczy piłki ani wyniku, mirroruje je z ramki otrzymanej od HOSTa.

Obie płytki, niezależnie od roli:

- mają aktywne wyjście VGA i renderują obraz lokalnie (własny monitor, własna kopia rozgrywki),
- liczą lokalnie WYŁĄCZNIE własną paletkę (moduł paddle_mover) na podstawie własnych przycisków - zero opóźnienia w sterowaniu,
- prowadzą lokalny automat stanu gry (game_fsm) zsynchronizowany z drugą płytką przez wymianę stanu (IDLE/READY/PLAYING/END) - obie strony muszą wcisnąć start (btnC), by rozpocząć rozgrywkę.

Komunikacja UART jest pełnoduplexowa, dwiema oddzielnymi ramkami w obu kierunkach jednocześnie:

HOST -> CLIENT (12 B, co ~1/60 s): pozycje obu paetek, pozycja piłki, wynik obu graczy, stan gry.

CLIENT -> HOST (4 B, co ~1/60 s): własna (lokalnie policzona) pozycja paetki CLIENTa oraz jego lokalny stan gry.

Piny: JC[0] = RX, JXADC[0] = TX - identycznie na obu płytkach. Płytki łączy się na krzyż (TX jednej do RX drugiej) plus wspólna masa (GND).

3. ZMIANY W KODZIE (repozytorium scalone do jednej gałęzi)

Nowe/zmienione moduły:

- rtl/game/paddle_mover.sv (nowy) - pozycja jednej, lokalnej paletki; zastępuje obie wcześniejsze wersje paddle_pos.sv.
- rtl/game/game_fsm.sv - wersja z wejściem peer_state, używana teraz na obu płytках (wcześniej istniała, ale nieużywana).
- rtl/uart/send_state_frame.sv / receive_state_frame.sv (przemianowane z send_bytes.sv / receive_bytes.sv, logika bez zmian) - kierunek HOST -> CLIENT.
- rtl/uart/send_paddle_frame.sv / receive_paddle_frame.sv (nowe) - kierunek CLIENT -> HOST.
- rtl/top_vga.sv oraz fpga/rtl/top_vga_basys3.sv - przepisane wokół sygnału roli is_host (z podwójnie synchronizowanego przełącznika sw_role), multiplexeru danych paletka/piłka/wynik oraz multiplexera ramek TX/RX.
- fpga/constraints/top_vga_basys3.xdc - dodany pin sw_role (SW0), włączony pin TX JXADC[0] (J3); VGA i btnC aktywne na stałe.
- Zaktualizowane pliki .prj i testbenche w sim/ pod nowe/przemianowane moduły i porty.

Repozytorium zamienione na jeden katalog projektu, jedna gałąź główna (main) zamiast master_new / player_2_new.

4. WERYFIKACJA

- synteza i implementacja zakończone bez błędów, bitstream wygenerowany poprawnie,
- timing się zgadza (dodatni WNS, 0 failing endpoints),
- Wszystkie ostrzeżenia (warningi) zgłoszone przez Vivado na etapach syntezy, implementacji oraz DRC zostały przeanalizowane. Są to standardowe, nieszkodliwe komunikaty narzędzia oraz advisory dotyczące istniejących wcześniej modułów, które nie mają żadnego wpływu na działanie wprowadzonych zmian ani na poprawność układu.

Napotkane problemy przy tej wersji projektu:

występuje problem z komunikacją po stronie HOST -> CLIENT

5. ZNANY, NIEROZWIĄZANY PROBLEM

Potwierdzona poprawność i weryfikacja na sprzęcie:

Uzgadnianie stanu gry (IDLE/READY/PLAYING/END): Stabilna komunikacja dwukierunkowa między płytkami. Węzeł HOST: Pełny przebieg rozgrywki działa prawidłowo (lokalna fizyka piłki, sterowanie paletką HOSTa oraz poprawny odbiór pozycji paletki CLIENTa w ramce 4-bajtowej). Węzeł CLIENT: Sterowanie lokalną paletką działa poprawnie. Dekodowanie ramek HOST -> CLIENT: Logika

dekodowania 12-bajtowej ramki jest poprawna. Zweryfikowano diagnostycznie (zliczanie bajtu STOP) oraz potwierdzono przy jednorazowym odblokowaniu łącza (jednoczesne wyświetlenie poprawnego stanu 9:3 oraz pozycji paletki na obu ekranach).

Opis usterki:

Podczas ciągłej pracy CLIENT nie odświeża pozycji piłki, paletki HOSTa ani wyniku (stan na ekranie zamraża się na wartościach początkowych). Jednorazowe odświeżenie obrazu przełączenie przełącznika roli (sw_role) na płytce HOSTa w stan CLIENT i powrót do HOST, czasami powoduje przestanie poprawnej ramki z danymi na temat paletki piłki i wyniku.

W rozwiązaniu usterki nie pomagały problemy z laptopem, który przy wgrywaniu bitstreama na płytke Basys3 przy części prób restartował się.

Wykluczone przyczyny:

Warstwa fizyczna: Wymiana przewodów oraz testy mechaniczne gniazd Pmod nie wpłynęły na objawy.

Buforowanie UART: Zwiększenie głębokości FIFO z 4 do 16 słów – brak poprawy.

Timing / Byterate: Zmniejszenie prędkości UART z ~19 200 do ~9600 baud – brak poprawy.

Resetowanie FSM: Cykliczne resetowanie FSM nadajnika/odbiornika (co ~200 ms) pogorszyło stabilność (całkowity brak aktualizacji).

Kolejna dodatkowa zmiana:

Dodanie stanu pośredniego: Poprawiono przejście w pliku receive_state_frame.sv tak, aby z BYTE_11 przechodzić do stanu BYTE_0 poprzez stan WAIT (tak samo, jak przy pozostałych bajtach). Ma to dać czas na ustabilizowanie się wskaźników FIFO.

Weryfikacja: Testy w symulacji potwierdziły, że po zmianie odbiornik stabilnie odbiera ciągły strumień klatek bez desynchronizacji.

Weryfikacja w praktyce:

Nie wiadomo dlaczego, ale nadal na płytkach nie działa to tak jak powinno. Obraz na hoście po staremu działa perfekcyjnie, ale obraz na kliencie w "zależności od humoru" raz pokazuje to samo co host (udokumentowane na krótkim video), raz kompletnie nie odbiera ramek z informacjami, a raz zaczyna pokazywać zupełnie co innego. Czasami lekkie docięnięcie SW0 na jednej z płytek generowało coś w stylu "zakłóceń": raz pokazywało faktyczny obraz, raz wracało do trybu "freeze", przez co nie potrafię stwierdzić czy wina leży po stronie fizycznej płytki, połączeniu, czy w kodzie.

Video z udokumentowanym po raz pierwszy działającym podejściem:

<https://drive.google.com/file/d/19gvpF02Ut36cNxQL-FA9qyCFK6O0R9wq/view?usp=sharing>

Zauważyłem, że po uruchomieniu gry po jednoczesnej zmianie Switchy SW0 na przeciwny stan i jeszcze raz na przeciwny (jak na początku) gra zaczyna działać poprawnie.

Kolejna próba naprawienia błędu:

Poprawka potwierdzania gotowości peera: W plikach `receive_state_frame.sv` oraz `receive_paddle_frame.sv` informacja o stanie gry drugiej strony (IDLE/READY/PLAYING/KONIEC) była zatwierdzana natychmiast po zobaczeniu samego bajtu nagłówka, zanim zweryfikowano poprawność całej reszty ramki (dane + bajt kończący). Dopasowanie samego nagłówka to tylko 4 bity — losowy/zakłócony bajt na łączy ma około 1 szansy na 16, żeby przypadkowo wyglądać jak nagłówek. Skutek: CLIENT potrafił uznać HOSTa za "gotowego" zanim ten faktycznie coś wysłał, przez co gra ruszała przedwcześnie z zamrożonym stanem. Poprawka: flaga stanu jest teraz buforowana i zatwierdzana dopiero razem z resztą pól, po potwierdzeniu całej ramki — dokładnie tak jak działo się to już wcześniej dla pozycji piłki i wyniku.

Identyczny błąd znaleziony po drugiej stronie: W `receive_paddle_frame.sv` (kierunek CLIENT→HOST) ostatni bajt ramki (terminator) wracał od razu do stanu szukania nagłówka, zamiast przez dodatkowy stan uspokajający — czyli dokładnie ten sam błąd, który wcześniej powodował zawieszanie się CLIENTa po jednej ramce (opisany w poprzedniej sekcji dla `receive_state_frame.sv`), tylko tym razem nienaprawiony po stronie odbioru potwierdzeń CLIENTa przez HOSTa. Poprawiony analogicznie.

Mechanizm samo-naprawy przy złym wyrównaniu ramki (RESYNC): Jeśli sprawdzenie odebranej ramki się nie zgadza, odbiornik (obie strony) celowo zjada jeden dodatkowy bajt przed ponownym szukaniem nagłówka, zamiast wracać dokładnie do tej samej, złej pozycji w strumieniu bajtów. Sam algorytm zweryfikowany osobną symulacją programową — zbiega niezawodnie przy realistycznych danych. Na sprzęcie nie rozwiązał problemu samodzielnie.

Sekwencja startowa (handshake) przed transmisją właściwych ramek: Obie płytki, zanim wyślą/odbiorą jakąkolwiek prawdziwą ramkę gry, najpierw ciągle wysyłają jeden stały bajt (0x55) i czekają, aż zobaczą go 8 razy z rzędu od drugiej strony — dopiero to potwierdza, że link jest nawiązany i obie strony faktycznie się widzą. Ma to wykluczyć sytuację, w której płytki (programowane przez Vivado nie w tej samej chwili) zaczynają nasłuchiwać w środku strumienia bajtów drugiej strony. Zweryfikowane na sprzęcie jako działające niezawodnie (obie płytki zawsze wykrywają się nawzajem) — ale problem z odbiorem ramki stanu przez CLIENTa nadal występuje nawet po tej poprawce.

Diagnostyka sprzętowa (diody LED): Dodano 9 diod diagnostycznych (`led[8:0]`) pokazujących na żywo: czy dana płytka wysyła cokolwiek, czy odbiera poprawną ramkę, czy w ogóle widzi coś przypominającego nagłówek, czy dochodzi do RESYNC, jaki jest zgłaszany stan gry drugiej strony, oraz czy sekwencja startowa się powiodła. Posłużyło to do systematycznego wykluczenia kolejnych hipotez.

Aktualny wniosek: Test zamiany ról (SW0 na obu płytkach) pokazał, że usterka wędruje za rolą CLIENTA, a nie za konkretną płytką ani przewodem — czyli to nie fizyczne uszkodzenie jednego konkretnego elementu. Jednocześnie zachowanie pozostaje loteryjne (czasem działa po restarcie, czasem nie) mimo wyeliminowania w kodzie wszystkich sensownych przyczyn logicznych (błędne wyrównanie na starcie, brak korekcji błędów, przedwczesne zatwierdzanie stanu). Prawdopodobnie jest to problem fizyczny (połączenie/kable)

Jak sprawić aby gra działała poprawnie:

Jak wyżej napisane, gra startuje niestety działająca jak "loteria": w losowych momentach przy restartach płytek. Aby włączyć działającą poprawnie grę, włączam obie płytki i obserwuję LDO na obu płytkach - czy obie migają. Jeśli nie, to klikam LEWY guzik od restartu (BTNL) na obu płytkach (np w tym samym czasie). Jeśli obie diody LED0 migają - gra jest w gotowości do startu.

